

qBOUNCE CMOS Neutron Detector

Pipeline & Live Acquisition — User Guide

August 26, 2026

Contents

1	What this software does	1
2	Installation on the offline Windows 10 machine	2
3	Standard workflow (Pipeline tab)	3
4	Parameters reference (Parameters tab)	3
4.1	Acquisition presets (IDS camera)	3
4.2	Detector parameters (C++)	4
4.3	Matched operating point (important)	4
4.4	Live parameters (Live tab)	5
4.5	Energy spectrum (Results tab)	6
4.6	Camera damage tester (radiation health)	6
5	Data products	7
6	Maintenance — modifying and redeploying the code	7
7	Troubleshooting	8

1 What this software does

A CMOS camera (IDS, 5472×3672 px, Mono8) watches a boron-10 converter layer. A neutron capture, $n + {}^{10}\text{B} \rightarrow \alpha + {}^7\text{Li}$, emits the two daughters *back-to-back*: only one of them reaches the sensor and leaves a bright, compact track (≥ 5 px). The software:

1. learns the per-pixel noise of the sensor from dark frames (*background model*: mean, σ , dead-pixel map);
2. detects clusters of pixels above $\text{mean} + n_\sigma \sigma$;
3. classifies each cluster with a machine-learning model (**lithium** = heavy track, **gamma**, **artifact**);
4. counts neutrons *live*: $N_{\text{neutrons}} = N_\alpha + N_{\text{Li}}$ (one daughter per capture);
5. estimates the deposited energy of each track (**total_charge** = $\sum_{\text{pixels}} (\text{ADU} - \text{background})$) and builds the $\alpha/{}^7\text{Li}$ energy spectrum;
6. monitors camera health (hot/dead pixels, radiation damage).

2 Installation on the offline Windows 10 machine

Step A — on any machine WITH internet (e.g. the development Mac):

```
python3 PrepareSetupLab.py
```

This builds `ExportLab/` — the complete offline kit: sources, GUI, docs, the trained classifier, every Python wheel for Windows (including `cmake` and `ninja` as pip wheels), the prebuilt OpenCV C++ package, and a generated `cpp_requirements.txt` manifest (parsed from `CMakeLists.txt`'s `find_package` calls — the closest thing to a “pipreqs for C++”).

Step B — on the offline lab machine: copy `ExportLab` to e.g. `C:\qBounce`. `WindowsLauncher.py` works fully offline (venv + wheels, installs `cmake/ninja` into the venv with no admin rights, extracts the bundled OpenCV and passes `-DOpenCV_DIR` to CMake, compiles the detector) and has three deliberately separate usages, so that the slow “provision the machine” work is not repeated every single day:

- `python WindowsLauncher.py -check` — verifies the setup **only** (~10s): no build, no GUI, nothing changed on disk. Run it once on arrival, or whenever something looks wrong.
- `python WindowsLauncher.py` — the **full run**: venv + packages + OpenCV + CMake build, then launches the GUI.
- `python WindowsLauncher.py -no-launch` — everything above *except* starting the GUI at the end (used by `LabValidation.py`, below, to provision unattended).

Run the full `WindowsLauncher.py` **once**. After that provisioning has succeeded, the **daily** start no longer goes through the launcher at all:

- `CheckSetup.bat` — double-click to re-run `-check` (verification only, safe to run any time something looks wrong).
- `RunLab.bat` — double-click for the **daily** start: runs `Code\venv\Scripts\python.exe PipelineController.py` directly from the already-provisioned venv, and refuses with a clear message if that venv does not exist yet (i.e. if `WindowsLauncher.py` was never run to completion).

Validating the kit: `python LabValidation.py` exercises the whole chain unattended — runs `WindowsLauncher.py -check`, then a full `-no-launch` provisioning, proves the resulting `detector(.exe)` actually **starts** (i.e. that the OpenCV DLLs resolve), and finally replays the exact commands the GUI's Build button would issue. Every subprocess has a timeout, so a run can never hang overnight waiting for input. Walk away and read `lab_validation_report.txt`, written next to the script, for a PASS/FAIL line per check; exit code 0 means every check the machine could run, passed.

Two things the kit cannot bundle: the **C++ compiler** and the **IDS peak SDK**.

- **Compiler:** must be **MSVC** (Visual Studio Build Tools — create an offline layout with `vs_buildtools.exe -layout` on an online PC if missing; the launcher detects and explains this). **MinGW is not a substitute:** the bundled OpenCV binaries are a `vc16` build, and MinGW cannot link against it. A machine with MinGW on `PATH` would otherwise look like it “has a compiler” and only fail later, at the link step, which is why the launcher checks specifically for MSVC rather than accepting any `g++`.
- **IDS peak SDK:** still install it from its own installer for real-camera capture (drivers + GenTL producer). Its Python *bindings*, however, are **not included in the standard installer** (verified on IDS peak 26.06, on a real machine) — the kit no longer expects to find them under `Program Files`. Instead `PrepareSetupLab.py` downloads `ids_peak` and

`ids_peak_ip1` straight from PyPI into `Code\wheels`, so the offline machine installs them like any other Python dependency. Only the Python layer travels with the kit — the native SDK (drivers/GenTL) must still be installed on the lab machine itself.

3 Standard workflow (Pipeline tab)

Run the steps top to bottom. Each step's output feeds the next.

1. **Clean** — deletes previous outputs (background model, detections CSVs, result plots). *Keeps* the hand-labelled annotation sample and the trained ML model.
2. **Build** — compiles the C++ detector (`cmake + make`).
3. **Background Model** — runs the detector over the *dark* folder. Produces `background_model.yml` (binary format: per-pixel mean, variance accumulator, sample count, dead-pixel mask). **Redo this whenever exposure, gain, temperature or the camera changes** — the model is specific to the acquisition conditions.
4. **Cluster Detection** — runs the detector over the *signal* folder (resuming from the background model) → `detections.csv`: one row per cluster with 20 features (position, size, charge, shape, SNR...).
5. **Labeling** — opens the annotation GUI on a sample of clusters. You assign the true class of each cluster (hotkeys shown on the buttons; [U]/**Ctrl-Z** undoes a mistake). Because α and ${}^7\text{Li}$ tracks cannot be told apart visually, both are labelled **lithium** (heavy track).
6. **Classify / Train** — trains a Random-Forest on your labelled sample (quality = cross-validated F1, shown in the GUI) and exports it as `model_bundle.joblib + model_bundle.onnx`. Predict then applies the model to the *whole* `detections.csv` → `detections_classified.csv` + summary plots. This is the offline neutron count of the run.
7. **Backtest** — re-runs the model on a *golden dataset* (a CSV with a human `true_label` column, chosen in **Parameters** → **Backtesting**) and reports precision/recall per class. Use an independently labelled sample, not the training sample, for an honest number.
8. **Live tab** — real-time acquisition and neutron counting (Section 4.4).

4 Parameters reference (Parameters tab)

4.1 Acquisition presets (IDS camera)

Parameter	Meaning and impact
Acquisition mode	Energy resolution (default): reduced exposure/gain so heavy tracks stay below the 255 saturation level → the deposited-energy spectrum is usable. High sensitivity : full exposure/gain (historical behaviour); use when faint events (gammas) matter more than energy resolution.
Exposure us (energy)	Exposure time of the energy preset. Tune until <code>study_energy</code> reports <i>Saturated</i> $\approx 0\%$. Shorter exposure = darker tracks, fewer per frame; note that counting efficiency is the <i>duty cycle</i> = exposure $\times$ fps — if you shorten the exposure, raise the fps accordingly or you will miss neutrons arriving between frames.

Gain dB (energy)	Analog gain of the energy preset. Start at 0.
Target saturated px/frame	The only knob of Auto-tune exposure : acceptable mean number of saturated pixels (≥ 254) per frame. It is an <i>absolute pixel count</i> , not a percentage: with ~ 40 tracks $\times \sim 10$ px each, a target of 50 still allows $\sim 10\%$ of track area to clip; use 10–20 for clean spectroscopy.
Auto-tune exposure (beam ON)	Halves the exposure from 500 ms until the target is met, then fills the energy-mode exposure field. Run it with the beam ON — it tunes on real track brightness. If it hits the 100 μ s floor, lower the gain and re-run.

4.2 Detector parameters (C++)

Parameter	Meaning and impact
n_sigma (default 5)	Detection threshold: a pixel is an event if $\text{ADU} > \text{mean} + n_\sigma \sigma$. Lower = more sensitive and more noise clusters; higher = misses faint tracks. <code>src/study_nsigma.py</code> plots cluster statistics vs. n_σ to choose it objectively.
warmup_frames	Number of dark frames used to build the dead-pixel map (auto = the whole dark folder).
Min cluster size (px) (default 4)	Clusters smaller than this are <i>dropped at the source</i> (no CSV row, no classification). Heavy tracks are ≥ 5 px; gammas/betas are 1–3 px. Set to 0 to record everything again (e.g. to label gammas). Dropping junk reduced $\sim 11\,000$ clusters/frame to ~ 60 on real data with no loss of heavy tracks.
Min cluster charge (ADU) (default 300)	Same idea on deposited charge. Heavy tracks carry $\gtrsim 700$ ADU; noise/artifacts $\lesssim 70$.
Cluster gap tolerance (px) (default 0)	Morphological closing radius that bridges small gaps in split tracks (useful for muons). 0 = off.
Cluster grow radius (px) (default 1)	After clustering, adds neighbouring pixels above Grow min SNR to the footprint: recovers the sub-threshold halo charge around a track (better energy proxy). Changing it changes the charge scale — redo the energy calibration afterwards.
Grow min SNR (default 1.0)	Minimum per-pixel SNR for grown pixels.

Two protections are automatic (no knob): pixels saturated/black during most of the warmup are masked, and any pixel that is an *event for 5 consecutive frames* is declared hot and masked permanently (real particles never hit the same pixel every frame) — this catches radiation damage that appears *during* a run.

4.3 Matched operating point (important)

The background model, the cluster filters and the trained classifier form a **matched set**, tied to one gain. If you lower the gain to de-saturate tracks for the energy spectrum, three things move at once: the noise σ (so **rebuild the background model** at the new gain), the absolute ADU scale (so use **Min peak SNR**, which is gain-invariant, not **Min cluster charge**), and the classifier’s input distribution — a model trained on 100%-saturated tracks has never seen a de-saturated one and may mislabel it. Three safeguards make this visible: (i) the model records the saturation it was trained at, and the live view pops a one-time *operating-point mismatch*

warning if the live saturation diverges by >25 points; (ii) **Check separability** (Results tab) tests whether signal and noise can still be told apart in gain-robust features and shows a red dialog if they have MERGED — at which point the neutron count is meaningless; (iii) the pre-filters are a throughput guardrail only (the classifier does the physics), set loose enough not to touch real tracks. The clean way to run de-saturated: pick a gain (**Auto-tune gain**), rebuild the background model, take a labelling sample *at that gain*, retrain, and keep the gain fixed — so the model always matches the saturation it was trained on.

4.4 Live parameters (Live tab)

The Live Source box only shows the fields relevant to the selected source: **Simulated folder/Simulated FPS** appear for the replay source and disappear for **IDS camera** (whose settings live in **Parameters**). Everything below the source box — **Duration**, batching, the model bundle, **Start/Stop** — applies identically to *either* source: there is one acquisition run, one Start control (the workflow’s ②), regardless of where frames come from.

Command-line note: the GUI always passes the source explicitly, but if `LiveDetectionEngine.py` is ever invoked directly (e.g. for debugging), `-camera {simulated,ids}` is **mandatory** — it has no default. This is deliberate: a stray invocation can never silently fall back to simulated frames when a real measurement was intended.

Parameter	Meaning and impact
Duration (s)	Total length of the live session (both sources); the engine auto-stops after this many seconds.
Batch seconds / Batch max frames	A <i>batch</i> is a display/classification grouping of frames (whichever limit is reached first closes the batch). It does not affect the physics — frames are streamed to the detector one by one regardless. With <code>Batch max frames = 1</code> you get one telemetry line per frame.
Rolling window (s)	Time window of the rolling neutron rate. Poisson statistics: the relative error shown ($\pm\%$) is $1/\sqrt{N}$ with N = neutrons in the window; $N \geq 100$ for $\sim 10\%$ precision.
Model bundle	The trained classifier; the live engine actually loads the <code>.onnx</code> + <code>.meta.json</code> exported next to it.
Save during live	What is written to disk while measuring, each on its own writer thread so detection is never throttled. raw frames : bit-identical <code>.bmp</code> archive of every camera frame (IDS-Cockpit replacement, ~ 20 MB/frame). data_analysed : compact products only (~ 20 KB/frame masked images + noise model + arrival map). Default: both.
Raw recording folder	Destination of the raw <code>.bmp</code> archive.

Storage budget (4 TB server, 20.1 MB/frame, 2 fps, 24 h/day).

mode	disk per day	4 TB lasts
raw frames	~ 3.5 TB	~ 1.2 days
data_analysed only	~ 4 GB	~ 1000 days

Caveat: `data_analysed` keeps only the pixels of clusters that passed the quality filters — it is an analysis product, not a raw archive. A lower detection threshold cannot be re-applied to it afterwards; keep raw frames (or lower the filters) whenever that flexibility matters.

Temporal coverage. Next to the fps choice, a coverage label shows exposure $\times$ fps — the fraction of wall-clock time the sensor integrates. Neutrons arriving outside it are never counted; divide any measured rate by the coverage. **Query camera timing** reads the *actual* exposure and resulting frame rate from the IDS camera (the sensor may clamp exposure below 1/fps for readout).

Guided lab session (workflow buttons). ① **Background model (beam OFF):** captures darks, builds the noise model and keeps a dated “before” copy. ② **Start measuring (beam ON):** live neutron counting. ③ **Camera health check (beam OFF):** fresh darks $\rightarrow$ “after” model $\rightarrow$ automatic radiation damage report. ④ **Background sanity:** classifies the beam-OFF clusters $\rightarrow$ the ML false-positive floor per frame, to subtract from any measured rate. The *ambient* room rate (neighbouring experiments) is measured by running ② with the beam OFF; ambient = that rate minus the ④ floor. ⑤ **End-of-experiment report:** one click after a run shows, in **Results**, the four maps of the experiment that just finished — *noise mean*, *noise σ* (the sensor’s per-pixel background), *arrival of all detected particles*, and *arrival of neutrons only* (positions of clusters classified lithium/alpha) — plus an automatic separability verdict (a red dialog appears only if signal and noise have MERGED). Each ② run records into its own timestamped folder (`data_analysed/live_<date>_<time>`), so ⑤ always refers to exactly the experiment just finished, even with several runs per day. ⑤ deliberately does *not* produce the energy spectrum: a normal 100–500-frame experiment has far too few heavy tracks for meaningful peaks — see the next paragraph.

Energy spectrum (office, large datasets). The spectrum button lives in **Parameters** $\rightarrow$ **Energy analysis**, directly above a path field pointing at a LARGE classified CSV (e.g. the rare 15 000-frame runs, or several sessions concatenated). Run it back at the office, not in the live lab loop; the result appears in **Results**.

Live telemetry. The statistics line shows cumulative neutrons ($= \alpha + \text{Li}$), per-channel counts and rates, neutrons/s, neutrons/frame and dead-pixel statistics. The **throughput line** shows *Camera fps* (frames actually delivered), *Analysed fps* (frames actually processed), and *Dropped*: green = pipeline keeps up; orange = queue filling, drops imminent; red = frames are being lost and *all displayed rates under-count reality by that fraction* — lower the camera fps or relax the analysis. At the end of a live run the detector writes `live_signal_arrival.tiff` (per-pixel capture counts; float32, ~ 80 MB) and a PNG preview shown in **Results** — it reveals *where* captures land on the sensor (e.g. a partially covered region).

4.5 Energy spectrum (Results tab)

Energy Spectrum runs `src/study_energy.py` on `detections_classified.csv`: a histogram of `total_charge` for heavy tracks. Expected physics: two strong lines (${}^7\text{Li}$ 0.84 MeV, α 1.47 MeV, ratio 1.75) plus two weak satellites (4% branch: 1.01 and 1.78 MeV). **Saturated pixels destroy energy information at capture time** — nothing can recover it in post-processing; the tool prints the saturated fraction, and only spectra with $\approx 0\%$ saturation (energy acquisition mode) are quantitative. With `-calibrate` the two fitted peaks are anchored to the known MeV lines, giving an absolute energy axis and the predicted satellite positions.

4.6 Camera damage tester (radiation health)

Give two background models — one built *before* and one *after* irradiation (browse buttons accept the binary `.yaml/.bgm` files; the workflow’s ①/③ buttons fill these in for you) — and **Run Degradation Analysis** reports the dark-current shift, hot-pixel counts on two independent signals, and a lifetime extrapolation.

Two hot-pixel signals are reported, deliberately: a raw **brightness threshold** on the mean dark current (fast, but sensitive to how many dark frames each model was built from — the “after” model is always built fresh, with no memory of the “before” one, so a small “after” sample gives a noisier mean and can flip a borderline pixel across the threshold either way by chance), and the **dead-pixel mask** each `background_model` run already builds internally from $\geq 95\%$ saturated/zero behaviour over its *entire* warmup, which cannot be tripped by a single noisy frame. Radiation damage is one-directional: a truly dead pixel cannot become alive again. If the dead-pixel count *drops* between “before” and “after”, that is not the sensor healing — it is the measurement noise floor of that particular before/after pair, and the tool says so explicitly. It also warns whenever the two models were built from very different frame counts (ratio < 0.5 or > 2), since that is exactly the condition that makes the brightness-threshold signal unreliable. Build “after” from as many fresh dark frames as time allows, ideally comparable to “before”.

5 Data products

File	Content
<code>background_model.yml</code>	Binary noise model (mean, variance, counts, dead mask).
<code>detections.csv</code>	One row per detected cluster, 20 features.
<code>detections_classified.csv</code>	Same + predicted label and confidence.
<code>data_analysed/...</code>	Compact archive: 5 raw sample frames, sparse masked PNGs ($\sim 1000\times$ smaller than raw — only pixels of valid clusters are kept), lossless float TIFFs of mean/ σ /arrival. Written by the offline steps and, when enabled, during live (<code>data_analysed/live_<date></code>) on a dedicated writer thread.
<code>live_recording/...</code>	Raw <code>.bmp</code> archive of every live frame, when raw recording is enabled.
<code>live_signal_arrival.tiff/.png</code>	Where captures landed during the last live run.
<code>energy_histogram.png</code>	Deposited-charge spectrum of heavy tracks.

6 Maintenance — modifying and redeploying the code

- **Python change** (`PipelineController.py` or `Code/src/*.py`): nothing to build — restart the GUI.
- **C++ change** (`/*.cpp/*.h`): recompile before the change takes effect: `cmake -build build -config Release` inside `Code` (or simply re-run `WindowsLauncher.py`, which rebuilds then launches). The GUI runs whatever `build/detector(.exe)` currently is — an old binary silently ignores new features.
- **New Python dependency**: re-run `PrepareSetupLab.py` on the online machine (it rescans imports with `pipreqs`, sanitises the list, re-downloads wheels) and copy the new `Code/wheels` over.
- **Redeploying to the lab**: re-run `PrepareSetupLab.py` $\rightarrow$ fresh `ExportLab/` $\rightarrow$ USB. For a small patch, copying just the changed `.py` files is enough (plus a rebuild if C++ changed). Never copy `venv/` or `build/` between machines.
- **Changed gain / camera settings**: rebuild the background model (①) at the new operating point, and if the saturation regime changed, re-label and retrain the classifier (see *Matched operating point*).

7 Troubleshooting

- *Neutron counts are zero*: retrain/re-export the classifier (Classify) so `model_bundle.onnx` exists; check the Console tab for engine errors.
- *Camera fps lower than requested*: the number shown is what the camera *actually delivers* (exposure time caps it at $1/\text{exposure}$; disk decoding caps the simulated camera).
- *Dropped frames (red)*: the analysis cannot keep up — reduce camera fps, or raise `Min cluster size/charge`.
- “Corrupted model file”: delete `background_model.yml` and rerun Background Model.
- *Arrival heatmap looks binary (0/255)*: normal at low statistics — each pixel has seen at most one capture; structure appears over long runs.